Universitatea Națională de Știință și Tehnologie Politehnica din București

Facultatea de Electronică, Telecomunicații și Tehnologia Informației

Pong game

Proiect Python

Nume student:

Chirițoiu Constantin-Sorin

Grupa: **423E**

1) Cerința

Acest proiect implementează jocul clasic Pong folosind biblioteca Pygame. Jocul permite celor doi jucători să controleze paletele și să concureze pentru a marca puncte, având posibilitatea de a pune pauză sau de a ieși din joc printr-un meniu grafic interactiv.

---

2) Descriere generală

Jocul Pong este un joc clasic de reflexe în care mingea se deplasează într-o arenă dreptunghiulară și este respinsă de paletele controlate de jucători. Scopul fiecărui jucător este de a marca puncte prin trimiterea mingii în partea opusă a terenului. Jocul include următoarele funcționalități:

- Controlul paletelor de către doi jucători, utilizând tastatura.
- O minge care își schimbă direcția la contactul cu marginea sau cu paletele.
- Sisteme de scor afișate pe ecran.
- Meniu cu butoane pentru pauză și ieșire.

---

3) Tehnologii utilizate

- **Python** - limbajul principal utilizat pentru implementarea logicii jocului.
- **Pygame** - bibliotecă utilizată pentru crearea interacțiunilor grafice și gestionarea evenimentelor.
- **Sys** - bibliotecă pentru ieșirea din aplicație.

Această combinație de tehnologii asigură o implementare fluidă și interactivă a jocului Pong.

---

4) Descrierea aplicației

Aplicația constă într-o fereastră principală în care sunt desenate elementele jocului: paletele, mingea, scorurile și meniul de pauză. Jocul este actualizat în timp real și permite o experiență competitivă pentru doi jucători.

5) Interfața jocului

➔ **Elemente principale:**

- ◆ Două palete colorate (roșie și albastră), controlate de jucătorii 1 și 2.
- ◆ O minge albă care se deplasează în arena de joc.
- ◆ Linie centrală albă pentru separarea terenului.

➔ **Meniu interactiv:**

- Două butoane funcționale:
  - **Pauză:** oprește temporar jocul.
  - **Ieșire:** închide aplicația.

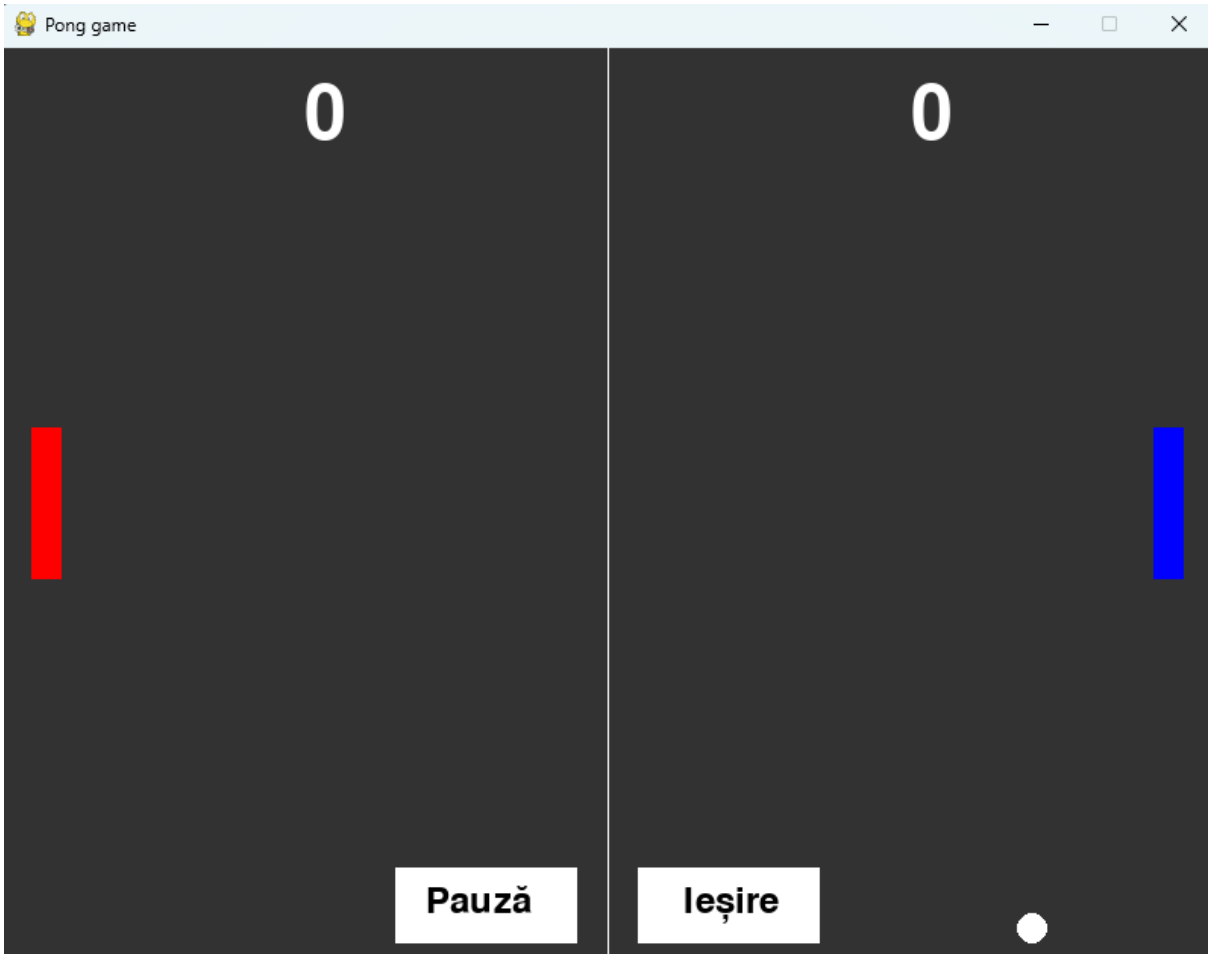

Figura 1. Prezentare Interfața Jocului

---

6) Prezentare funcționalități

Mișcarea paletelor:

- Paleta **jucătorului 1** este controlată cu tastele W (sus) și S (jos).
- Paleta **jucătorului 2** este controlată cu tastele Săgetă sus(↑) și Săgetă jos(↓).

Figura 2. Palete

Mișcarea mingii:

- Mingea se deplasează în linie dreaptă până la coliziunea cu o margine, o paletă sau ieșirea din teren.
- Coliziunile determină inversarea direcției mingii.

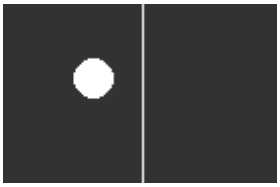

Figura 3. Minge

Sistemul de scor:

- Fiecare jucător primește 1 punct atunci când mingea trece de paleta adversară.
- Scorurile sunt afișate într-un format mare, vizibil, în partea de sus a ecranului.

Figura 4. Afișare Scoruri

Meniul jocului:

- Butoanele **Pauză** și **Ieșire** sunt poziționate în partea de jos a ecranului.
- Acțiuni posibile:
    - Apăsarea pe **Pauză** oprește temporar jocul și afișează un mesaj „Joc Pauzat”.
    - Apăsarea pe **Ieșire** închide aplicația.

Figura 5. Meniu Joc

7) Codul principal

- **Inițializarea jocului:**

  - Se definesc dimensiunile ferestrei, culorile, paletele, mingea și pozițiile lor inițiale.

```python
import pygame
import sys

pygame.init() # Inițializare pygame

LATIME_ECRAN = 800 # Dimensiuni ecran
INALTIME_ECRAN = 600

ALB = (255, 255, 255) # Culori
NEGRU = (0, 0, 0)
ROSU = (255, 0, 0)
ALBASTRU = (0, 0, 255)
GRI = (50, 50, 50)

LATIME_MINGE = 20 # Setări minge
INALTIME_MINGE = 20
VITEZA_MINGE_X = 4
VITEZA_MINGE_Y = 4

LATIME_PALETA = 20 # Setări palete
INALTIME_PALETA = 100
VITEZA_PALETA = 6

ecran = pygame.display.set_mode((LATIME_ECRAN, INALTIME_ECRAN)) # Inițializare ecran
pygame.display.set_caption("Pong game")

ceas = pygame.time.Clock() # Ceas pentru controlul ratei cadrelor

minge_x = LATIME_ECRAN // 2 - LATIME_MINGE // 2 # Poziția și viteza mingii
minge_y = INALTIME_ECRAN // 2 - INALTIME_MINGE // 2
viteza_minge_x = VITEZA_MINGE_X
viteza_minge_y = VITEZA_MINGE_Y

jucator1_x = 20 # Poziția paletelor
jucator1_y = INALTIME_ECRAN // 2 - INALTIME_PALETA // 2

jucator2_x = LATIME_ECRAN - 20 - LATIME_PALETA
jucator2_y = INALTIME_ECRAN // 2 - INALTIME_PALETA // 2

scor_jucator1 = 0 # Scoruri
scor_jucator2 = 0

font_mare = pygame.font.Font(None, 74) # Fonturi
font_mic = pygame.font.Font(None, 36)

latime_buton = 120 # Butoane pentru pauză și ieșire
inaltime_buton = 50
buton_pauza = pygame.Rect(LATIME_ECRAN // 2 - latime_buton - 20, INALTIME_ECRAN - inaltime_buton - 10, latime_buton, inaltime_buton)
buton_iesire = pygame.Rect(LATIME_ECRAN // 2 + 20, INALTIME_ECRAN - inaltime_buton - 10, latime_buton, inaltime_buton)

```

Figura 6. Inițializarea jocului

- **Actualizarea jocului:**

  - Se gestionează evenimentele utilizatorului (apăsări de taste sau click-uri).
  - Se actualizează pozițiile mingii și paletelor.
  - Se verifică coliziunile pentru a modifica traiectoria mingii sau pentru a actualiza scorurile.

```python
rulat = True # Bucle joc
pauza = False
while rulat:
    # Gestionare evenimente
    for eveniment in pygame.event.get():
        if eveniment.type == pygame.QUIT:
            rulat = False
        if eveniment.type == pygame.MOUSEBUTTONDOWN:
            if buton_pauza.collidepoint(eveniment.pos):
                pauza = not pauza
            if buton_iesire.collidepoint(eveniment.pos):
                rulat = False

    if not pauza:
        # Preluare taste
        taste = pygame.key.get_pressed()

        # Mișcare jucător 1
        if taste[pygame.K_W] and jucator1_y > 0:
            jucator1_y -= VITEZA_PALETA
        if taste[pygame.K_S] and jucator1_y < INALTIME_ECRAN - INALTIME_PALETA:
            jucator1_y += VITEZA_PALETA

        # Mișcare jucător 2
        if taste[pygame.K_UP] and jucator2_y > 0:
            jucator2_y -= VITEZA_PALETA
        if taste[pygame.K_DOWN] and jucator2_y < INALTIME_ECRAN - INALTIME_PALETA:
            jucator2_y += VITEZA_PALETA

        # Mișcare minge
        minge_x += viteza_minge_x
        minge_y += viteza_minge_y

        # Coliziune minge cu marginea de sus și jos
        if minge_y <= 0 or minge_y >= INALTIME_ECRAN - INALTIME_MINGE:
            viteza_minge_y = -viteza_minge_y

        # Coliziune minge cu paletele
        if (
            minge_x <= jucator1_x + LATIME_PALETA
            and jucator1_y < minge_y + INALTIME_MINGE
            and minge_y < jucator1_y + INALTIME_PALETA
        ) or (
            minge_x + LATIME_MINGE >= jucator2_x
            and jucator2_y < minge_y + INALTIME_MINGE
            and minge_y < jucator2_y + INALTIME_PALETA
        ):
            viteza_minge_x = -viteza_minge_x

        # Minge în afara limitelor
        if minge_x < 0:
            scor_jucator2 += 1
            minge_x = LATIME_ECRAN // 2 - LATIME_MINGE // 2
            minge_y = INALTIME_ECRAN // 2 - INALTIME_MINGE // 2
            viteza_minge_x = VITEZA_MINGE_X
            viteza_minge_y = VITEZA_MINGE_Y
        if minge_x > LATIME_ECRAN:
            scor_jucator1 += 1
            minge_x = LATIME_ECRAN // 2 - LATIME_MINGE // 2
            minge_y = INALTIME_ECRAN // 2 - INALTIME_MINGE // 2
            viteza_minge_x = -VITEZA_MINGE_X
            viteza_minge_y = -VITEZA_MINGE_Y
```

Figura 7. Actualizarea jocului

- **Interfața grafică:**
    - Se desenează elementele jocului la fiecare cadru.
    - Se afișează scorurile și meniul interactiv.

```python
    # Desenare elemente
    ecran.fill(GRI)

    # Desenare palete și minge
    pygame.draw.rect(ecran, ROSU, (jucator1_x, jucator1_y, LATIME_PALETA, INALTIME_PALETA))
    pygame.draw.rect(ecran, ALBASTRU, (jucator2_x, jucator2_y, LATIME_PALETA, INALTIME_PALETA))
    pygame.draw.ellipse(ecran, ALB, (minge_x, minge_y, LATIME_MINGE, INALTIME_MINGE))

    # Linie centrală
    pygame.draw.aaline(ecran, ALB, (LATIME_ECRAN // 2, 0), (LATIME_ECRAN // 2, INALTIME_ECRAN))

    # Scoruri
    text_jucator1 = font_mare.render(str(scor_jucator1), True, ALB)
    text_jucator2 = font_mare.render(str(scor_jucator2), True, ALB)
    ecran.blit(text_jucator1, (LATIME_ECRAN // 4, 20))
    ecran.blit(text_jucator2, (LATIME_ECRAN * 3 // 4, 20))

    # Desenare butoane
    pygame.draw.rect(ecran, ALB, buton_pauza)
    pygame.draw.rect(ecran, ALB, buton_iesire)
    text_pauza = font_mic.render("Pauză", True, NEGRU)
    text_iesire = font_mic.render("Ieșire", True, NEGRU)
    ecran.blit(text_pauza, (buton_pauza.x + 20, buton_pauza.y + 10))
    ecran.blit(text_iesire, (buton_iesire.x + 30, buton_iesire.y + 10))

    # Suprapunere pentru pauză
    if pauza:
        text_pauza = font_mare.render("Pauză Joc", True, ALB)
        ecran.blit(text_pauza, (LATIME_ECRAN // 2 - text_pauza.get_width() // 2, INALTIME_ECRAN // 2 - text_pauza.get_height() // 2))

    # Actualizare ecran
    pygame.display.flip()

    # Limitare rată cadre
    ceas.tick(60)
```

Figura 8. Interfața grafică

- **Închiderea jocului:**
    - Se gestionează oprirea jocului și închiderea aplicației folosind pygame.quit().

```
# Ieșire joc
pygame.quit()
sys.exit()
```

Figura 9. Închiderea jocului